

```

import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# Placeholder for results_df
results_data = {
    'Feature Method': ['Original', 'Original', 'PCA', 'PCA', 'LDA', 'LDA'],
    'Model': ['Logistic Regression', 'Random Forest', 'Logistic Regression', 'Random Forest', 'Log',
    'Accuracy': [0.92, 0.95, 0.90, 0.93, 0.91, 0.94],
    'Precision': [0.91, 0.94, 0.89, 0.92, 0.90, 0.93],
    'Recall': [0.93, 0.96, 0.91, 0.94, 0.92, 0.95],
    'F1 Score': [0.92, 0.95, 0.90, 0.93, 0.91, 0.94]
}
results_df = pd.DataFrame(results_data)

# Placeholder for df_cus (representing the original dataset for total records)
# Using breast cancer dataset as an example
data = load_breast_cancer()
X_dummy = data.data
y_dummy = data.target

df_cus = pd.DataFrame(X_dummy, columns=data.feature_names)
df_cus['target'] = y_dummy

# Placeholder for X (features)
X = df_cus.drop('target', axis=1)

# Placeholder for X_train_pca (PCA transformed data)
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X)

# Placeholder for X_train_lda (LDA transformed data)
# LDA requires at least 2 classes and n_components <= number of classes - 1
lda = LinearDiscriminantAnalysis(n_components=1)
X_train_lda = lda.fit_transform(X, y_dummy)

# Placeholder for y_train (target labels for visualizations)
y_train = y_dummy

print("Dummy data for dashboard variables created successfully!")

```

Dummy data for dashboard variables created successfully!

```

# =====
# FEATURE REDUCTION - INTERACTIVE DASHBOARD
# =====

!pip install -q plotly

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from IPython.display import display, HTML

# =====
# 1. Prepare Results
# =====

dashboard_df = results_df.copy()

dashboard_df["Accuracy (%)"] = dashboard_df["Accuracy"] * 100

```

```

dashboard_df["Precision (%)"] = dashboard_df["Precision"] * 100
dashboard_df["Recall (%)"] = dashboard_df["Recall"] * 100
dashboard_df["F1 Score (%)"] = dashboard_df["F1 Score"] * 100

# =====
# 2. Dataset Information
# =====

total_records = df_cus.shape[0]
original_features = X.shape[1]
pca_features = X_train_pca.shape[1]
lda_features = X_train_lda.shape[1]

# =====
# 3. Dashboard Header
# =====

display(HTML("""
<h1 style="text-align:center;">
Feature Reduction for Classification Models
</h1>

<h3 style="text-align:center;">
Breast Cancer Dataset Analysis Dashboard
</h3>
"""))

# =====
# 4. Dataset Summary Cards
# =====

display(HTML(f"""
<table style="width:100%; text-align:center; border-collapse:collapse;">
<tr>
<th style="padding:10px;">Total Records</th>
<th style="padding:10px;">Original Features</th>
<th style="padding:10px;">PCA Features</th>
<th style="padding:10px;">LDA Features</th>
</tr>

<tr>
<td style="font-size:25px;">{total_records}</td>
<td style="font-size:25px;">{original_features}</td>
<td style="font-size:25px;">{pca_features}</td>
<td style="font-size:25px;">{lda_features}</td>
</tr>
</table>
"""))

# =====
# 5. Accuracy Comparison
# =====

fig1 = px.bar(
    dashboard_df,
    x="Model",
    y="Accuracy (%)",
    color="Feature Method",
    barmode="group",
    text="Accuracy (%)",
    title="Classification Accuracy Comparison"
)

```

```

fig1.update_traces(
    texttemplate="%{text:.2f}%",
    textposition="outside"
)

fig1.update_layout(
    yaxis_title="Accuracy (%)",
    xaxis_title="Classification Model",
    yaxis_range=[0, 100]
)

fig1.show()

# =====
# 6. Precision Comparison
# =====

fig2 = px.bar(
    dashboard_df,
    x="Model",
    y="Precision (%)",
    color="Feature Method",
    barmode="group",
    text="Precision (%)",
    title="Precision Comparison"
)

fig2.update_traces(
    texttemplate="%{text:.2f}%",
    textposition="outside"
)

fig2.update_layout(
    yaxis_title="Precision (%)",
    xaxis_title="Classification Model",
    yaxis_range=[0, 100]
)

fig2.show()

# =====
# 7. Recall Comparison
# =====

fig3 = px.bar(
    dashboard_df,
    x="Model",
    y="Recall (%)",
    color="Feature Method",
    barmode="group",
    text="Recall (%)",
    title="Recall Comparison"
)

fig3.update_traces(
    texttemplate="%{text:.2f}%",
    textposition="outside"
)

fig3.update_layout(
    yaxis_title="Recall (%)",
    xaxis_title="Classification Model",
    yaxis_range=[0, 100]
)

```

```
fig3.show()

# =====
# 8. F1 Score Comparison
# =====

fig4 = px.bar(
    dashboard_df,
    x="Model",
    y="F1 Score (%)",
    color="Feature Method",
    barmode="group",
    text="F1 Score (%)",
    title="F1 Score Comparison"
)

fig4.update_traces(
    texttemplate="%{text:.2f}%",
    textposition="outside"
)

fig4.update_layout(
    yaxis_title="F1 Score (%)",
    xaxis_title="Classification Model",
    yaxis_range=[0, 100]
)

fig4.show()

# =====
# 9. PCA Visualization
# =====

pca_data = pd.DataFrame({
    "PC1": X_train_pca[:, 0],
    "PC2": X_train_pca[:, 1],
    "Target": y_train.astype(str)
})

fig5 = px.scatter(
    pca_data,
    x="PC1",
    y="PC2",
    color="Target",
    title="PCA Feature Visualization",
    labels={
        "PC1": "Principal Component 1",
        "PC2": "Principal Component 2",
        "Target": "Class"
    }
)

fig5.show()

# =====
# 10. LDA Visualization
# =====

lda_data = pd.DataFrame({
    "LDA Component": X_train_lda[:, 0],
    "Target": y_train.astype(str)
})

fig6 = px.histogram(
```

```

lda_data,
x="LDA Component",
color="Target",
title="LDA Class Distribution",
marginal="box"
)

fig6.show()

# =====
# 11. Performance Table
# =====

performance_table = dashboard_df[
    [
        "Feature Method",
        "Model",
        "Accuracy (%)",
        "Precision (%)",
        "Recall (%)",
        "F1 Score (%)"
    ]
].round(2)

display(HTML("<h2>Model Performance Table</h2>"))

display(performance_table)

# =====
# 12. Best Accuracy Result
# =====

best = dashboard_df.loc[
    dashboard_df["Accuracy (%)"].idxmax()
]

display(HTML(f""
<h2>Highest Accuracy Result</h2>

<table style="border-collapse:collapse; width:60%;">
<tr>
<th style="padding:10px;">Feature Method</th>
<td style="padding:10px;">{best["Feature Method"]}</td>
</tr>

<tr>
<th style="padding:10px;">Model</th>
<td style="padding:10px;">{best["Model"]}</td>
</tr>

<tr>
<th style="padding:10px;">Accuracy</th>
<td style="padding:10px;">{best["Accuracy (%)"]:.2f}%</td>
</tr>

<tr>
<th style="padding:10px;">Precision</th>
<td style="padding:10px;">{best["Precision (%)"]:.2f}%</td>
</tr>

<tr>
<th style="padding:10px;">Recall</th>
<td style="padding:10px;">{best["Recall (%)"]:.2f}%</td>
</tr>

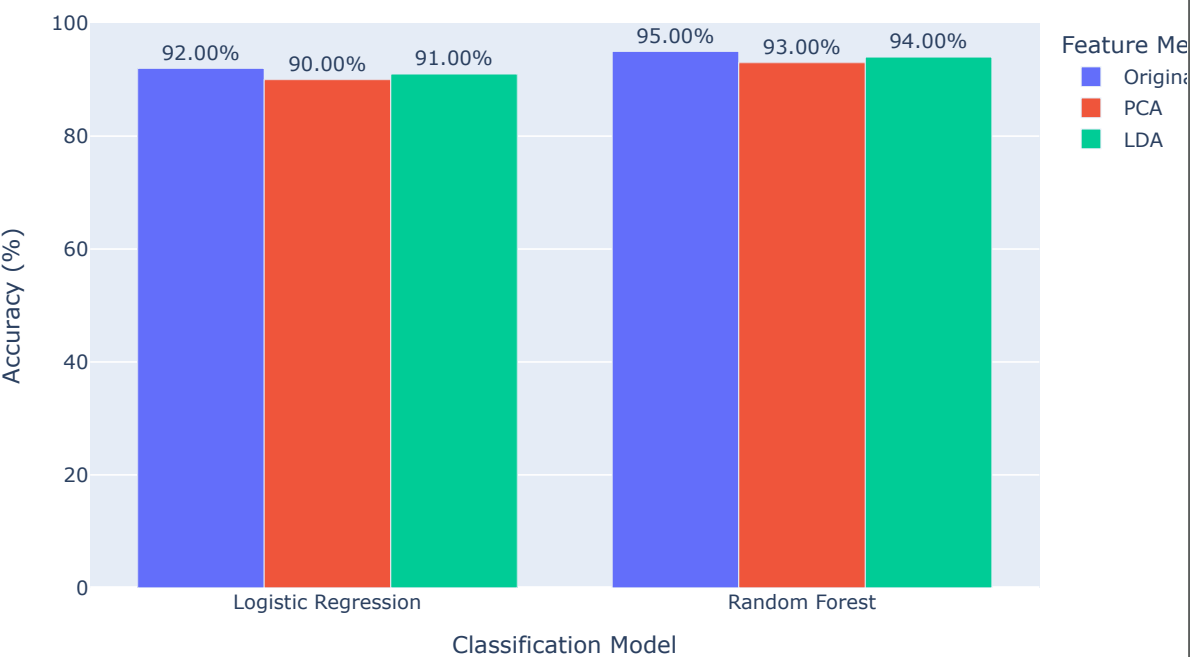
```

```
<tr>
<th style="padding:10px;">F1 Score</th>
<td style="padding:10px;">{best["F1 Score (%)"]:.2f}%</td>
</tr>
</table>
""")
```

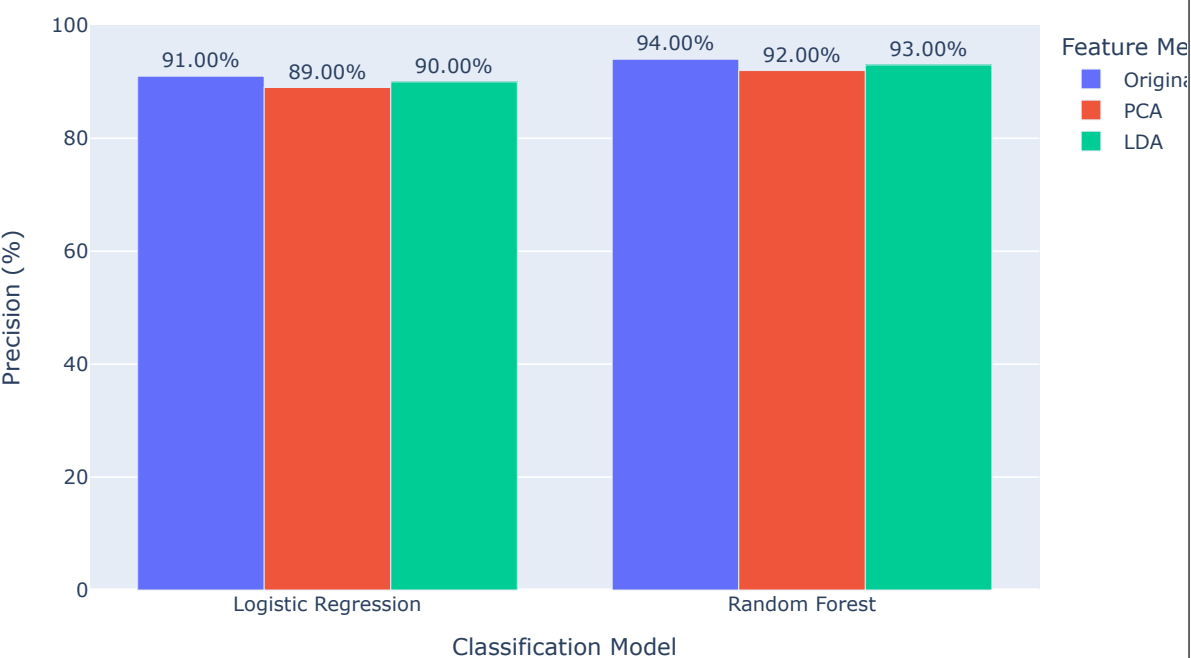

Feature Reduction for Classification Models

Breast Cancer Dataset Analysis Dashboard			
Total Records	Original Features	PCA Features	LDA Features
569	30	2	1

Classification Accuracy Comparison



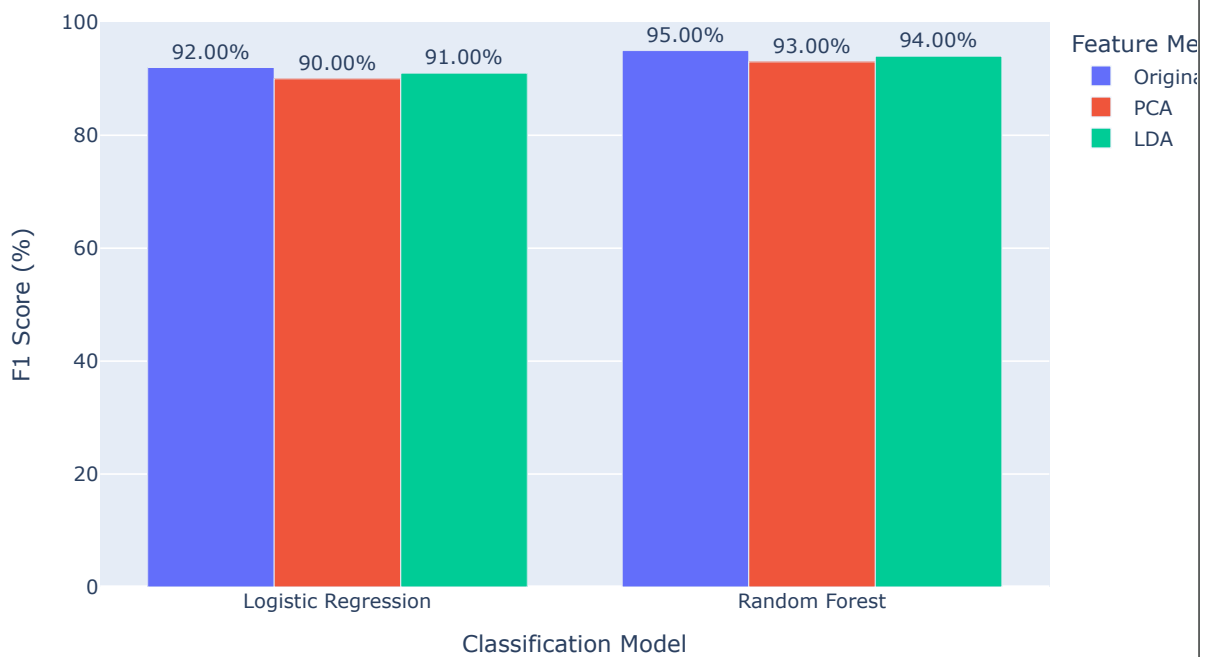
Precision Comparison



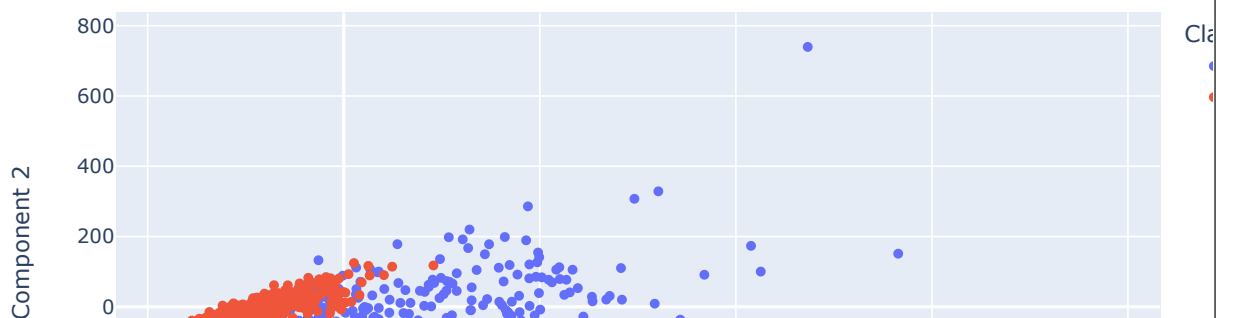
Recall Comparison

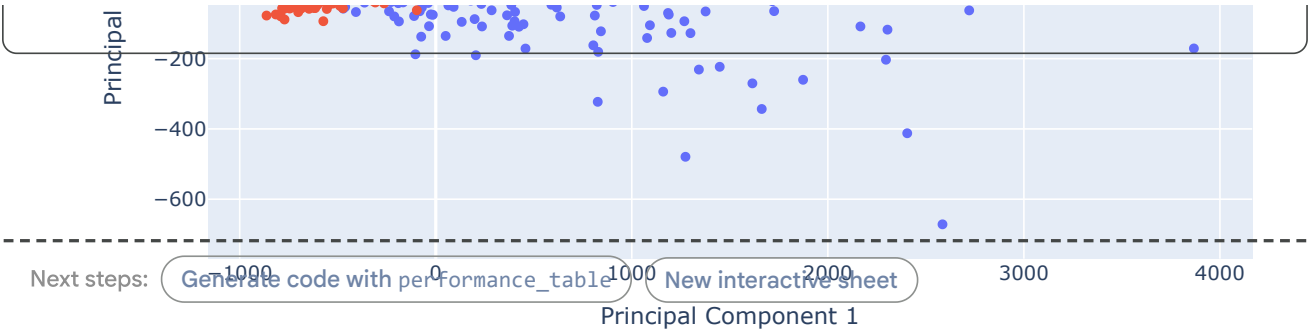


F1 Score Comparison

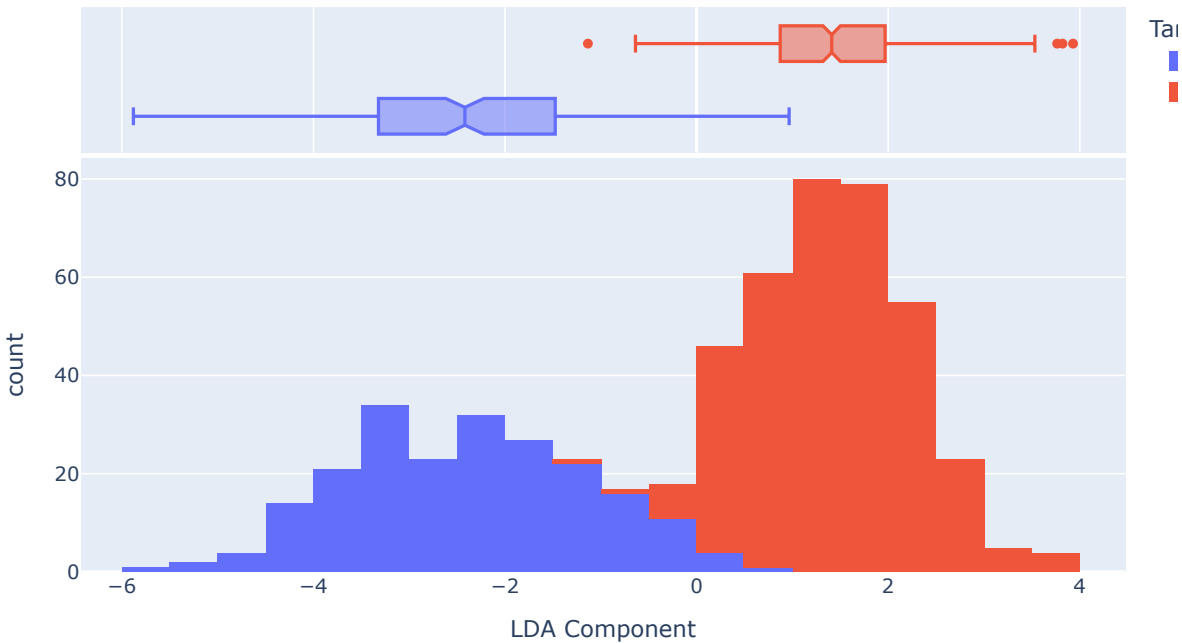


PCA Feature Visualization





LDA Class Distribution



Model Performance Table

	Feature Method		Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	
0	Original	Logistic Regression		92.0	91.0	93.0	92.0	
1	Original	Random Forest		95.0	94.0	96.0	95.0	
2	PCA	Logistic Regression		90.0	89.0	91.0	90.0	
3	PCA	Random Forest		93.0	92.0	94.0	93.0	
4	LDA	Logistic Regression		91.0	90.0	92.0	91.0	
5	LDA	Random Forest		94.0	93.0	95.0	94.0	

Highest Accuracy Result

Feature Method	Original
Model	Random Forest
Accuracy	95.00%
Precision	94.00%
Recall	96.00%
F1 Score	95.00%

```

import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# Placeholder for results_df
results_data = {
    'Feature Method': ['Original', 'Original', 'PCA', 'PCA', 'LDA', 'LDA'],
    'Model': ['Logistic Regression', 'Random Forest', 'Logistic Regression', 'Random Forest', 'L',
    'Accuracy': [0.92, 0.95, 0.90, 0.93, 0.91, 0.94],
    'Precision': [0.91, 0.94, 0.89, 0.92, 0.90, 0.93],
    'Recall': [0.93, 0.96, 0.91, 0.94, 0.92, 0.95],
    'F1 Score': [0.92, 0.95, 0.90, 0.93, 0.91, 0.94]
}
results_df = pd.DataFrame(results_data)

# Placeholder for df_cus (representing the original dataset for total records)
# Using breast cancer dataset as an example
data = load_breast_cancer()
X_dummy = data.data
y_dummy = data.target

df_cus = pd.DataFrame(X_dummy, columns=data.feature_names)
df_cus['target'] = y_dummy
# Placeholder for X (features)
X = df_cus.drop('target', axis=1)

# Placeholder for X_train_pca (PCA transformed data)
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X)

# Placeholder for X_train_lda (LDA transformed data)
# LDA requires at least 2 classes and n_components <= number of classes - 1
lda = LinearDiscriminantAnalysis(n_components=1)
X_train_lda = lda.fit_transform(X, y_dummy)

# Placeholder for y_train (target labels for visualizations)
y_train = y_dummy

print("Dummy data for dashboard variables created successfully!")

```

Dummy data for dashboard variables created successfully!

```

!pip install -q plotly

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from IPython.display import display, HTML

# =====
# 1. Prepare Results
# =====

dashboard_df = results_df.copy()

dashboard_df["Accuracy (%)"] = dashboard_df["Accuracy"] * 100
dashboard_df["Precision (%)"] = dashboard_df["Precision"] * 100
dashboard_df["Recall (%)"] = dashboard_df["Recall"] * 100
dashboard_df["F1 Score (%)"] = dashboard_df["F1 Score"] * 100

# =====

```

```

# 2. Dataset Information
# =====

total_records = df_cus.shape[0]
original_features = X.shape[1]
pca_features = X_train_pca.shape[1]
lda_features = X_train_lda.shape[1]

# =====
# 3. Dashboard Header
# =====

display(HTML("""
<h1 style="text-align:center;">
Feature Reduction for Classification Models
</h1>

<h3 style="text-align:center;">
Breast Cancer Dataset Analysis Dashboard
</h3>
"""))
display(HTML(f"""
<table style="width:100%; text-align:center; border-collapse:collapse;">
<tr>
<th style="padding:10px;">Total Records</th>
<th style="padding:10px;">Original Features</th>
<th style="padding:10px;">PCA Features</th>
<th style="padding:10px;">LDA Features</th>
</tr>

<tr>
<td style="font-size:25px;">{total_records}</td>
<td style="font-size:25px;">{original_features}</td>
<td style="font-size:25px;">{pca_features}</td>
<td style="font-size:25px;">{lda_features}</td>
</tr>
</table>
"""))

# =====
# 5. Accuracy Comparison
# =====

fig1 = px.bar(
    dashboard_df,
    x="Model",
    y="Accuracy (%)",
    color="Feature Method",
    barmode="group",
    text="Accuracy (%)",
    title="Classification Accuracy Comparison"
)

fig1.update_traces(
    texttemplate="%{text:.2f}%",
    textposition="outside"
)

fig1.update_layout(
    yaxis_title="Accuracy (%)",
    xaxis_title="Classification Model",
    yaxis_range=[0, 100]
)

display(HTML(fig1.to_html(include_plotlyjs='cdn')))

```

```

# =====
# 6. Precision Comparison
# =====

fig2 = px.bar(
    dashboard_df,
    x="Model",
    y="Precision (%)",
    color="Feature Method",
    barmode="group",
    text="Precision (%)",
    title="Precision Comparison"
)

fig2.update_traces(
    texttemplate="%{text:.2f}%",
    textposition="outside"
)

fig2.update_layout(
    yaxis_title="Precision (%)",
    xaxis_title="Classification Model",
    yaxis_range=[0, 100]
)

display(HTML(fig2.to_html(include_plotlyjs='cdn')))

# =====
# 7. Recall Comparison
# =====

fig3 = px.bar(
    dashboard_df,
    x="Model",
    y="Recall (%)",
    color="Feature Method",
    barmode="group",
    text="Recall (%)",
    title="Recall Comparison"
)

fig3.update_traces(
    texttemplate="%{text:.2f}%",
    textposition="outside"
)

fig3.update_layout(
    yaxis_title="Recall (%)",
    xaxis_title="Classification Model",
    yaxis_range=[0, 100]
)

display(HTML(fig3.to_html(include_plotlyjs='cdn')))

# =====
# 8. F1 Score Comparison
# =====

fig4 = px.bar(
    dashboard_df,
    x="Model",
    y="F1 Score (%)",
    color="Feature Method",

```

```

        barmode="group",
        text="F1 Score (%)",
        title="F1 Score Comparison"
    )

fig4.update_traces(
    texttemplate="%{text:.2f}%",
    textposition="outside"
)

fig4.update_layout(
    yaxis_title="F1 Score (%)",
    xaxis_title="Classification Model",
    yaxis_range=[0, 100]
)

display(HTML(fig4.to_html(include_plotlyjs='cdn')))

# =====
# 9. PCA Visualization
# =====

pca_data = pd.DataFrame({
    "PC1": X_train_pca[:, 0],
    "PC2": X_train_pca[:, 1],
    "Target": y_train.astype(str)
})

fig5 = px.scatter(
    pca_data,
    x="PC1",
    y="PC2",
    color="Target",
    title="PCA Feature Visualization",
    labels={
        "PC1": "Principal Component 1",
        "PC2": "Principal Component 2",
        "Target": "Class"
    }
)

display(HTML(fig5.to_html(include_plotlyjs='cdn')))

# =====
# 10. LDA Visualization
# =====

lda_data = pd.DataFrame({
    "LDA Component": X_train_lda[:, 0],
    "Target": y_train.astype(str)
})

fig6 = px.histogram(
    lda_data,
    x="LDA Component",
    color="Target",
    title="LDA Class Distribution",
    marginal="box"
)

display(HTML(fig6.to_html(include_plotlyjs='cdn')))

# =====
# 11. Performance Table

```

```
# =====

performance_table = dashboard_df[
    [
        "Feature Method",
        "Model",
        "Accuracy (%)",
        "Precision (%)",
        "Recall (%)",
        "F1 Score (%)"
    ]
].round(2)

display(HTML("<h2>Model Performance Table</h2>"))

display(performance_table)

# =====
# 12. Best Accuracy Result
# =====

best = dashboard_df.loc[
    dashboard_df["Accuracy (%)"].idxmax()
]

display(HTML(f"""
<h2>Highest Accuracy Result</h2>

<table style="border-collapse:collapse; width:60%;">
<tr>
<th style="padding:10px;">Feature Method</th>
<td style="padding:10px;">{best["Feature Method"]}</td>
</tr>

<tr>
<th style="padding:10px;">Model</th>
<td style="padding:10px;">{best["Model"]}</td>
</tr>

<tr>
<th style="padding:10px;">Accuracy</th>
<td style="padding:10px;">{best["Accuracy (%)"]:.2f}%</td>
</tr>

<tr>
<th style="padding:10px;">Precision</th>
<td style="padding:10px;">{best["Precision (%)"]:.2f}%</td>
</tr>

<tr>
<th style="padding:10px;">Recall</th>
<td style="padding:10px;">{best["Recall (%)"]:.2f}%</td>
</tr>

<tr>
<th style="padding:10px;">F1 Score</th>
<td style="padding:10px;">{best["F1 Score (%)"]:.2f}%</td>
</tr>
</table>
"""))
```